

Generación y Animación Computacional: El Atractor de Clifford

Documentación Técnica y Metodología

1. Introducción a los Atractores

Un atractor representa el conjunto de estados (o puntos) hacia los cuales un sistema dinámico tiende a evolucionar con el tiempo, sin importar cuáles sean las condiciones iniciales exactas. En el contexto de la teoría del caos y la dinámica no lineal, los atractores “extraños” exhiben geometría fractal y una sensibilidad extrema a las condiciones iniciales.

El atractor utilizado en esta animación es el **Atractor de Clifford**, descubierto y popularizado por el matemático Clifford Pickover a principios de la década de 1990. Este sistema surge del mapeo iterativo de funciones trigonométricas (seno y coseno) sobre un plano bidimensional. Las ecuaciones que describen su comportamiento están definidas por:

$$x_{n+1} = \sin(ay_n) + c \cos(ax_n) \quad (1)$$

$$y_{n+1} = \sin(bx_n) + d \cos(by_n) \quad (2)$$

donde a, b, c y d son constantes matemáticas. Alterar estos parámetros cambia fundamentalmente la forma en la que la órbita de los puntos se pliega sobre sí misma.

Fuente de validación: Para explorar más sobre la teoría detrás de estos mapeos, es altamente recomendable el texto fundacional *Nonlinear Dynamics and Chaos* de Steven H. Strogatz.

2. Implementación en Python (Las Ecuaciones)

Para implementar este sistema matemático de manera eficiente, utilizamos la biblioteca **NumPy**. Dado que necesitamos procesar decenas de miles de puntos para crear textura, inicializamos y preasignamos memoria en los

arreglos numéricos (arrays) en lugar de usar listas dinámicas de Python puras, optimizando sustancialmente el rendimiento computacional.

```
import numpy as np

# 1. Definición de parametros iniciales
a, b, c, d = -1.4, 1.6, 1.0, 0.7
n_points = 50000

# 2. Preasignación de memoria
x = np.zeros(n_points)
y = np.zeros(n_points)

# Condiciones iniciales de la trayectoria
x[0], y[0] = 0.1, 0.1

# 3. Bucle iterativo computando las ecuaciones de Clifford
for i in range(1, n_points):
    x[i] = np.sin(a * y[i-1]) + c * np.cos(a * x[i-1])
    y[i] = np.sin(b * x[i-1]) + d * np.cos(b * y[i-1])
```

Documentación Oficial: Se utiliza `numpy.zeros` para la asignación estructurada de memoria y el módulo matemático integrado `numpy.sin` / `numpy.cos`.

3. Integración en un Gráfico Estático

Para visualizar el resultado de las ecuaciones computadas, integramos los arreglos numéricos resultantes en un gráfico de dispersión bidimensional (*scatter plot*) empleando **Matplotlib**.

La estética de “arte matemático” se logra mapeando cada coordenada (x_i, y_i) con un tamaño de marcador diminuto (`s=0.2`) y una opacidad sumamente baja (`alpha=0.3`). Esto simula un renderizado basado en densidad: las regiones donde la trayectoria matemática se cruza frecuentemente acumulan color y se vuelven brillantes, mientras que las visitas esporádicas forman una estela difuminada.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 6), facecolor='black')
ax.scatter(x, y, s=0.2, color='cyan', alpha=0.3, edgecolors='none')
ax.axis('off') # Se eliminan los ejes cartesianos por
               # estetica
plt.show()
```

Documentación Oficial: Parámetros estéticos de `matplotlib.pyplot.scatter`.

4. Integración Dinámica (La Animación)

La animación no consiste en dibujar una figura nueva desde cero por cada cuadro (lo cual sería ineficientemente lento). En su lugar, se envuelve la lógica iterativa dentro de un bucle de tiempo y se actualiza en memoria únicamente el arreglo de datos del objeto `PathCollection` (el gráfico de dispersión), utilizando la clase `FuncAnimation`.

4.1. Mutación de Parámetros

Para que la geometría del atractor mute fluidamente de un cuadro a otro, los parámetros matemáticos a y b dejan de ser escalares estáticos para convertirse en funciones continuas del tiempo (t).

Se estableció una animación a 60 cuadros (frames). El tiempo abstracto avanza de 0 a 2π creando un bucle perfecto (loop ininterrumpido):

- $t = \left(\frac{\text{frame}}{60}\right) \times 2\pi$
- $a(t) = -1,4 + 0,2 \sin(t)$
- $b(t) = 1,6 + 0,2 \cos(t)$

El uso desfasado de $\sin$ y $\cos$ (una rotación circular) garantiza que los parámetros cambien gradualmente orbitando alrededor de sus valores base en el espacio de parámetros, provocando que los filamentos del atractor se doblen y reorganicen suavemente.

```
import matplotlib.animation as animation

# El scatter se instancia vacío originalmente
scat = ax.scatter([], [], s=0.2, color='cyan', alpha=0.3)

# La función se ejecuta por cada cuadro iterativo
def update(frame):
    t = (frame / 60.0) * 2 * np.pi
    a = -1.4 + 0.2 * np.sin(t)
    b = 1.6 + 0.2 * np.cos(t)

    # Recalculamos la trayectoria entera usando el nuevo 'a'
    # y 'b'
    for i in range(1, n_points):
        x[i] = np.sin(a * y[i-1]) + c * np.cos(a * x[i-1])
        y[i] = np.sin(b * x[i-1]) + d * np.cos(b * y[i-1])

    # Inyectamos los nuevos arreglos calculados al objeto de
    # Matplotlib
```

```
    scat.set_offsets(np.c_[x, y])
    return scat,

# blit=True optimiza la carga computacional dibujando solo lo
# actualizado
ani = animation.FuncAnimation(fig, update, frames=60, blit=
    True)
```

Documentación Oficial: Para profundizar en el motor de animación y el parámetro `blit`, consultar la documentación de [matplotlib.animation.FuncAnimation](#).